

ENEE 467: Robotics Project Laboratory

Dynamic Programming and Tabular Reinforcement Learning

Lab 8

Learning Objectives

The purpose of this lab is to provide a first hands-on experience with reinforcement learning using the Gymnasium framework. After completing this lab, students should be able to:

1. Implement model-based RL algorithms: value iteration and policy iteration, to understand the RL loop, convergence, and exploration-exploitation tradeoffs.
2. Implement model-free RL algorithms: Q-Learning and SARSA, to understand temporal-differencing, tune hyperparameters and compare performances of various RL models.

Reinforcement learning (RL) provides a framework for sequential decision-making in which an agent evaluates actions based on their long-term consequences. In the tabular setting, this involves computing value functions, improving policies, and analyzing state-action returns using explicit state and action enumerations. This lab focuses on these foundations through hands-on implementation of dynamic programming and classical tabular RL algorithms, providing the conceptual and practical foundation for RL in continuous-space environments and deep RL.

Related Reading

- I. R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction" (Second Edition), MIT Press, 2018. [Open Access](#).
- II. Lecture 14, "Introduction to Reinforcement Learning," ENEE 467: Robotics Project Laboratory, University of Maryland, College Park, MD, Nov. 2025.

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. This lab contains no standalone questions. Rather, there are several lab **Exercises** after the main environment setup. Work through the implementation steps for each exercise and answer the accompanying questions. Complete the exercises during the lab and include your solutions in your lab writeup.

List of Exercises

Exercise 1. Random Policy for FrozenLake (15 pts.)	5
Exercise 2. Tabular Model-based RL (30 pts.)	5
Exercise 3. Tabular Model-free RL (45 pts.)	7

1 Lab Procedure

1.1 Software Installation

To prepare your system for the labs, follow the steps outlined in this section to install the necessary software. We will use a command-line interface (CLI) for all installations. Your machine should be running Ubuntu 20.04 or a newer version. This ensures compatibility with the required libraries and drivers. If you're not on a compatible system, consider using a virtual machine (e.g., VirtualBox, VMware, WSL2) or the UMIACS computing service (recommended).

1.1.1 Accessing the UMIACS Computing Service (Nexus)

If you are enrolled in ENEE 467, you should have received an email confirming your UMIACS class account for accessing the Nexus cluster. A detailed setup guide is available on the [UMIACS Wiki](#); we summarize only the essentials here. The lab computers already have an SSH client and the GlobalProtect VPN client setup. Start the GlobalProtect client if you haven't already, and connect to the **TunnelAll gateway**. Then verify that you have an SSH client installed using:

```
ssh -V
```

This should return the OpenSSH version installed on your machine. You can then connect to a Nexus cluster using your UMD Directory ID (`<umd_id>`) by running the following command:

```
ssh -Y <umd_id>@nexusclass.umiacs.umd.edu
```

On your first connection, you will be asked to verify the server's identity. Enter yes and press Enter to store the server's host key in your `~/.ssh/known_hosts` file. After this initial confirmation, you will not be prompted again for that host unless its key changes. Once added, you will then be prompted to log in with your UMD credentials within the terminal session. Enter your UMD passphrase and press Enter to access the cluster. Conduct the rest of the procedure within an active remote session. Note that you will be prompted to log in with your UMD credentials each time you access the cluster or if your session disconnects.

1.1.2 Getting this Lab's Code

From your current remote session, create a **Labs** folder within your home directory, move to it, and clone the lab repository (ignore the askpass Gtk-WARNING):

```
mkdir ~/Labs && cd ~/Labs  
git clone https://github.com/ENEE467-F2025/lab-8.git
```

1.1.3 RL Environment Setup

To set up your remote machine for the RL labs, we recommend using a **conda** environment that can be installed using [Miniconda](#) as follows. From a terminal within the remote machine, move to your home directory using `cd ~`, then run:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

Next, run the downloaded shell script using:

```
bash Miniconda3-latest-Linux-x86_64.sh
```

Confirm each prompt, and allow the installation to complete. When it's finished, restart your current shell by running `exec $SHELL`. You should then see the base environment activated:

```
(base) bash-4.4
```

Deactivate it using the following command:

```
conda deactivate
```

Then create a new `conda` environment named `rl467` and activate it by running the following:

```
export CONDA_PLUGINS_AUTO_ACCEPT_TOS="yes"
conda create --name rl467 python=3.11 -y
conda activate rl467
```

Continue by installing the `Gymnasium` library for RL tasks:

```
conda install -c conda-forge swig -y
pip install gymnasium[box2d]
```

1.1.4 Editing Files with Vim

On remote machines without GUI-based editors, terminal text editors provide an effective alternative. One such tool is `Vim`, which you can start in your remote session using:

```
vim
```

To simultaneously create and open a new file (say `a.txt`), call `vim` with the name of the file:

```
vim a.txt
```

This opens an empty Vim buffer in the terminal. By default you cannot type, so press `Shift+ A` to enter INSERT mode. Copy and paste work as in the Bash shell (e.g., `Ctrl+Shift+c`, `Ctrl+Shift+v`). Vim offers many editing shortcuts (see the [Vim Cheat Sheet](#)), but we will skip them. After editing, exit with `Ctrl+C`, then type `:wq` and press Enter to save and close Vim (in contrast, entering `:qa` + Enter will close Vim without saving the file).

1.2 Test Your Installation



Figure 1: LunarLander and FrozenLake environments in Gymnasium.

Create a new Python file named `test_gym.py` in the `lab-8` repo directory:

```
cd ~/Labs/lab-8
vim test_gym.py
```

Copy-paste the following code template into Vim:

```
# Turn off ALSA warnings
import os
os.environ["SDL_AUDIODRIVER"] = "dummy"

import gymnasium as gym # main library providing sandboxes and RL utilities

import warnings
warnings.filterwarnings("ignore", category=UserWarning, module="pygame.pkgdata")

# Initialize the environment
env = gym.make("LunarLander-v3", render_mode="human")
print("Action space:", env.action_space)
print("Observation space:", env.observation_space)

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward
    # and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

Cross-check the file's indentation, save it, then run it using the following commands:

```
echo 'export SDL_AUDIODRIVER=dummy' >> ~/.bashrc
source ~/.bashrc
conda activate rl467 && python3 ~/Labs/lab-8/test_gym.py
```

You should see the text: “Action space: Discrete(4) ... Observation space: Box....”

- 🐞 a. If you encounter the integrated-graphics error ‘libGL error: MESA-LOADER: failed to open iris’, install:

```
conda install -c conda-forge libstdcxx-ng -y
```

- b. To reactivate the `rl467` environment in a new or reconnected session, run:

```
source ~/miniconda3/bin/activate && conda activate rl467
```

Exercise 1. Random Policy for FrozenLake (15 pts.)

To familiarize yourself with the Gymnasium-style API, the code snippet in `test_gym.py` implements a random policy that uniformly samples actions in the LunarLander environment. Complete this exercise by performing the following steps:

1. Modify the code to reflect the `FrozenLake` environment by changing the environment ID.
2. What are the action and observation space dimensions? (**Hint:** In `test_gym.py`, print `env.action_space.n` and `env.observation_space.n`.)
3. Provide a short screen recording (of at most 10 seconds) that shows the outcomes of *two episodes* in the `FrozenLake` environment when using the random policy. (**Hint:** An episode ends in the `FrozenLake` environment if the agent (elf) falls into the lake or reaches the goal (gift box); see [Figure 1](#)).

Note that Gymnasium provides many environments to explore, and you are encouraged to experiment with additional ones available at [Gymnasium](#).

Exercise 2. Tabular Model-based RL (30 pts.)

In this exercise, you will implement tabular model-based RL algorithms—value iteration and policy iteration—for a grid-world setting using dynamic programming. The provided file `dynamic_programming.py` contains the skeletal code for the `FrozenLake` environment where you will implement the algorithms. The tasks to be completed are as follows:

1. Complete the `compute_action_values` function (marked with a `TODO`) so that it returns the correct action values, $Q(s, a)$, stored in the NumPy array `action_values`, using Bellman's equation (see p. 17 of Ref. II in [Related Reading](#)). For reference, Bellman's equation is:

$$V^\pi(s) = \sum_a \pi(a | s) \overbrace{\sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]}^{Q(s, a)}, \quad (1)$$

where $\pi(a | s)$ gives the probability of selecting action a in state s . In the code, we treat a as an index drawn from the discrete action set of size `n_actions`, i.e. $a \in \{0, 1, \dots, \text{n_actions}-1\}$. Thus, given the current state `state` (i.e., s), action index `a`, discount factor `gamma` (i.e., γ), and the value function V^π stored in the array `V` (of length `n_states`), the transition probability $p(s', r | s, a)$ (`prob`), next state s' (`next_state`), and reward r (`reward`) needed to compute `action_values` may be obtained via:

```
prob, next_state, reward, _ = env.unwrapped.P[state][a]
```

Finally, note that `compute_action_values` computes only $Q(s, a)$ (the blue term in (1)) for each action index; the policy $\pi(a | s)$ is applied later during policy evaluation. This keeps the function generic for both value iteration and policy iteration.

2. Complete the code for both value iteration (VI) and policy iteration (PI) by modifying the `TODO`-highlighted blocks in the methods `value_iteration` and `policy_iteration`. Refer to the reading material (Ref. II in [Related Reading](#), pg. 20 (VI); pgs. 26-28 (PI))

for pseudocode, noting that the `theta` argument represents ϵ from the lecture notes, k indexes the set $[0, \text{max_iterations})$, s indexes the state space S , with cardinality `n_states`, and $V_{k+1}(s)$ is the action value computed from the set returned by the helper function `compute_action_values` from item 1 above.

3. Provide a short screen recording (of at most 10 seconds) showing the agent acting under the learned policy.
4. Change the `FrozenLake` environment to have stochastic transitions: set the parameter `slippery` to `True` in the `if __name__ == '__main__':` block.
5. By filling in code within the `TODO`-highlighted block in the `compute_action_values` function, modify the computed reward (stored in the `reward` variable) by:
 - (a) adding a positive scalar value to the reward at each step, e.g., $r' = r + 1$.
 - (b) multiplying the reward at each step by a scalar value, e.g., $r' = r * 2$.
 - (c) adding a penalty for each step, e.g., $r' = r - 0.1$.

Questions

1. Compare how the value function and policy change for each reward modification.
2. Does the optimal policy change? Why or why not?
3. How does each reward modification affect the convergence of each algorithm?
4. How do deterministic vs. stochastic environments (i.e., setting `slippery=True`) influence algorithm performance?

Fix for ALSA Driver Warnings

If you see any `ALSA` driver warnings, you may safely ignore them, as they do not affect execution. If you prefer to suppress these messages on the remote lab machines, set the `SDL_AUDIODRIVER` environment variable before running your programs:

```
export SDL_AUDIODRIVER=dummy
python3 <python_program>.py
```

Note that this workaround is intended only for the remote machines; using it on a system with proper audio drivers may cause issues.

Exercise 3. Tabular Model-free RL (45 pts.)

In this exercise, you will implement tabular model-free RL algorithms—Q-Learning (off-policy) and SARSA (on-policy)—for a grid-world setting. The provided file `tabular_rl.py` contains the skeletal code for the `FrozenLake` environment where you will implement the algorithms, plot the results and tune the hyperparameters. The tasks to be completed are as follows:

1. Fill in the code for updating the Q-function in the training loop of the `q_learning` function.
2. Similar to the style of `q_learning`, create a new function `sarsa`, which has the modified Q update according to the SARSA update rule.
3. Plot the evolution of rewards over timesteps or episodes, and compare the performance (plots) of the algorithms. Change the environment `seed` and plot the results. Are the results same across different `seed` values?
4. Change the values of the hyperparameters and show the corresponding rewards-vs-time plots.
5. The current code implements a constant ϵ -greedy action selection strategy. Modify this with your own ϵ -decay schedule (e.g., linear decay, exponential decay, step decay, etc.). Plot the results of rewards-vs-time for your choice(s).
6. Provide visualizations of the learned policy.
7. Change the `FrozenLake` environment to have stochastic transitions by setting `is_slippery=True` in the *first* gym.make return statement of the `create_env` function.
8. Repeat the steps for `Taxi` and `CliffWalking` environment, by changing the environment IDs.

Questions

1. Which of the algorithms has a stable learning curve?
2. How do the hyperparameters affect the performances of the policies?
3. How does stochasticity affect the performances of the policies?
4. In the algorithms implemented so far, we explicitly represent a table for the Q-function and value function. What limitations might this approach face in larger or continuous environments?

2 Grading Rubric

Component	Points
Lab Report and Formatting	10
Exercise 1.	15
Exercise 2.	30
Exercise 3.	45
Total	100